

INTERACTIVE LEARNING STUDIO • PYTHON

Object-Oriented Programming, built from the blueprint up.

A complete, hands-on tutorial that takes you from "what is a class?" to designing real, extensible Python systems — with working code, live exercises, and four graded checkpoints along the way.

// THE SHAPE OF A CLASS, AT A GLANCE

Animal

- name
- sound
- + speak()

—inherits→

Dog(Animal)

- + speak() **overridden**
- + fetch()

EST. TIME

70–90 minutes

PREREQUISITES

- Python variables, functions, loops
- Comfort reading basic scripts

YOU'LL BE ABLE TO

- Design classes & objects
- Model relationships with inheritance
- Apply encapsulation & polymorphism
- Use dunder methods & composition

REWARD SYSTEM

Earn a badge per module + a Mastery Certificate at the end.

● Foundations Badge

● Inheritance Badge

● Application Badge

● Mastery Badge



Module 01 Foundations — Classes & Objects

Beginner

What is a class, really?

A **class** is a blueprint for creating objects. It doesn't do anything by itself — it just describes what data (*attributes*) and behavior (*methods*) every object built from it will have. An **object** (or *instance*) is one concrete thing made from that blueprint.

ANALOGY

Think of a class as a cookie cutter and objects as the cookies. The cutter defines the shape (attributes) and how you can decorate it (methods), but each cookie is separate dough — changing one cookie's icing doesn't change another's.

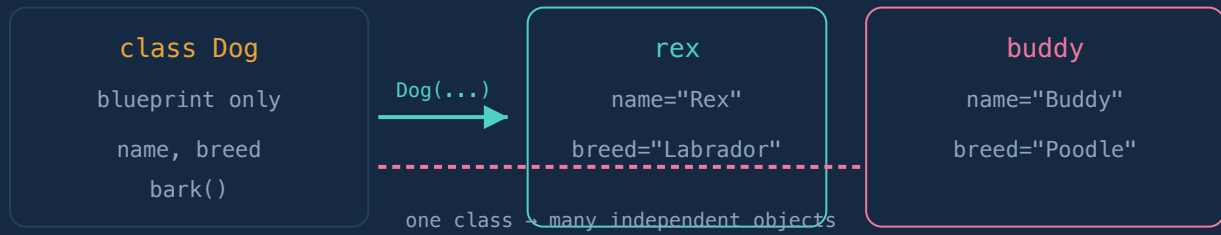
```
class Dog:
    # the blueprint
    def __init__(self, name, breed):
        self.name = name        # attribute
        self.breed = breed      # attribute

    def bark(self):             # method
        return f"{self.name} says Woof!"

rex = Dog("Rex", "Labrador")   # rex is an object/instance
print(rex.bark())              # Rex says Woof!
```

The role of `__init__` and `self`

`__init__` is the **constructor** — Python calls it automatically the moment you create an object, so you can set up its starting attributes. `self` is simply "the object currently being worked on"; every instance method receives it automatically as the first parameter so it knows which object's data to use.



Class attributes vs. instance attributes

INSTANCE ATTRIBUTE	CLASS ATTRIBUTE
Set in <code>__init__</code> via <code>self.x = ...</code>	Defined directly in the class body
Unique to each object	Shared by all objects unless overridden
<code>self.name</code>	<code>Dog.species = "Canine"</code>

COMMON MISCONCEPTION

Beginners often think `self` is a keyword. It's not — it's just a convention. You could name it anything, but every experienced Python developer expects `self`, so deviating breaks readability for no benefit.

Try it: build an object mentally

Pick values and see the equivalent Python object print out.

Rex · Labrador

Milo · Beagle

Luna · Husky

Click a button above.

Exercise 1.1 — Write a class

Write a `Book` class with attributes `title` and `author`, and a method `describe()` that returns `"{title} by {author}"`.

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def describe(self):
        return f'{self.title} by {self.author}'
```

Key takeaways

- A class is a blueprint; an object is a concrete instance of that blueprint.
- `__init__` runs automatically to set up a new object's attributes.
- `self` refers to "this particular object" inside a method.
- Instance attributes belong to one object; class attributes are shared.

Checkpoint Quiz — Foundations

4 questions · instant feedback · 70%+ to unlock Module 2

1. What does a class primarily represent in Python?

A single value stored in memory

A blueprint describing attributes and methods for objects

A loop that repeats code

A file on disk

Why: A class is a blueprint — it defines what data and behavior its objects will have, but doesn't itself hold that data until an object is created.

2. What is the purpose of `__init__`?

It deletes an object

It's called automatically to set up a new object's starting attributes

It converts an object to a string

It defines class-level constants only

Why: `__init__` is the constructor — Python calls it the moment an object is created, so you can initialize its attributes.

3. In `def bark(self):`, what does `self` refer to?

A required Python keyword with special syntax meaning

The class itself, not any particular object

The specific object the method is being called on

The parent class

Why: `self` is just a (strongly conventional) parameter name referring to the specific instance the method was called on — it's not a reserved keyword.

4. Which is true about instance attributes vs class attributes?

Instance attributes are shared by all objects

Class attributes are unique per object

Instance attributes are typically set via `self.x` in `__init__`, unique per object

There is no difference in Python

Why: Instance attributes (`self.x = ...`) belong to one specific object; class attributes are defined in the class body and shared unless overridden.

4/4 (100%)

Nice work — you've cleared this checkpoint.

[Unlock Module 2 →](#)Module
02

Building Relationships — Inheritance

[Beginner →](#)
[Intermediate](#)

Inheritance: reusing and specializing behavior

Inheritance lets one class (the *child* or *subclass*) reuse and extend the attributes and methods of another (the *parent* or *superclass*). It models an "is-a" relationship: a Dog *is an* Animal.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return f"{self.name} makes a sound."

class Dog(Animal):
    # Dog inherits from Animal
    def speak(self):
        # overriding the parent method
        return f"{self.name} says Woof!"

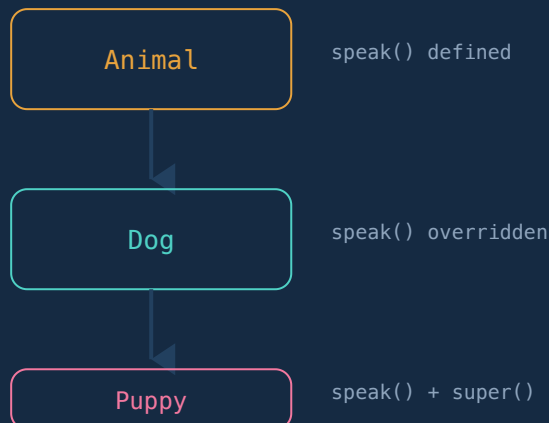
class Puppy(Dog):
    def speak(self):
        parent_line = super().speak() # call the parent's version too
        return parent_line + " (but it's more of a yip)"
```

ANALOGY

Inheritance is like a family recipe passed down through generations. A grandchild's recipe starts from grandma's base recipe (the parent class) but can tweak an ingredient (override a method) or add a new step (add a new method) — while still keeping most of the original intact.

Method Resolution Order (MRO)

When you call a method, Python looks for it starting at the object's own class, then walks up the chain of parents until it finds a match. `super()` lets a child explicitly call the parent's version instead of fully replacing it.



WITHOUT INHERITANCE	WITH INHERITANCE
Copy-paste <code>spea</code> , <code>eat</code> , <code>sleep</code> into every animal class	Define once in <code>Animal</code> , reuse in every subclass
Bug fixes must be repeated everywhere	Fix once in the parent, all children benefit
No shared "type" relationship	<code>isinstance(rex, Animal)</code> is <code>True</code>

COMMON MISCONCEPTION

Overriding a method does not delete the parent's version — it just means Python finds the child's version first. The parent version still exists and can be reached with `super()`.

Try it: who speaks?

Animal

Dog

Puppy

Click a class to see what `spea`() returns.

Exercise 2.1 — Extend the hierarchy

Create a `Cat` class that inherits from `Animal` and overrides `spea`() to return "`{name}` says Meow!".

```
class Cat(Animal):  
    def speak(self):  
        return f"{self.name} says Meow!"
```

Key takeaways

- Inheritance models "is-a" relationships and lets subclasses reuse parent code.
- Overriding replaces a method's behavior in the subclass; `super()` lets you still call the parent's version.
- Python resolves method calls by walking the class hierarchy (MRO).
- Deep, tangled hierarchies can become hard to follow — inherit only when there's a genuine "is-a" relationship.

Checkpoint Quiz — Inheritance

4 questions · instant feedback · 70%+ to unlock Module 3

1. What relationship does inheritance model?

"has-a"

"is-a"

"uses-a"

"knows-a"

Why: Inheritance models an "is-a" relationship: a Dog is an Animal, so it can reuse and extend Animal's behavior.

2. What happens when a subclass overrides a parent method?

The parent method is permanently deleted

Python finds the subclass version first, but the parent version still exists and can be reached with `super()`

Both versions run automatically every time

It causes a compile error

Why: Overriding doesn't erase the parent's method — it just means the subclass's version is found first. `super()` lets you still invoke the parent's version explicitly.

3. What does `super().speak()` do inside a subclass method?

Calls a random method

Explicitly calls the parent class's version of `speak()`

Deletes the current object

Creates a brand-new object

Why: `super()` gives you a reference to the parent class so you can call its original implementation, often to extend rather than fully replace it.

4. Which best describes Method Resolution Order (MRO)?

The order attributes are printed in

The order Python searches classes (starting from the object's own class upward) to find a method

A random search order

The order classes are imported in a file

Why: MRO is the order Python walks the class hierarchy — starting with the instance's own class and moving up through its parents — to resolve a method call.

4/4 (100%)

Nice work — you've cleared this checkpoint.

[Unlock Module 3 →](#)

Module 03 Practical Application — Encapsulation & Polymorphism

Intermediate

Encapsulation: protecting internal state

Encapsulation means bundling data and the methods that operate on it together, and controlling how that data is accessed or changed from outside. Python signals "private" intent with a leading underscore (convention, not enforced) or a double underscore (name-mangled, harder to access accidentally).

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance        # "private" attribute

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    @property
    def balance(self):                  # controlled, read-only access
        return self.__balance

acc = BankAccount(100)
acc.deposit(50)
print(acc.balance)    # 150 — but acc.__balance directly is blocked
```

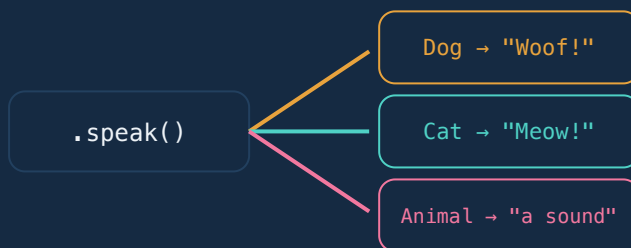
ANALOGY

Encapsulation is like a vending machine. You interact through buttons (public methods) — you never reach in and rearrange the snacks yourself (internal attributes). The machine controls exactly how its internal state can change.

Polymorphism: one interface, many behaviors

Polymorphism means different classes can be used interchangeably if they share a common interface — the same method name behaves appropriately for each type.

```
for creature in [Dog("Rex"), Cat("Tom"), Animal("???")]:  
    print(creature.speak())  # each object responds in its own way  
  
# Rex says Woof!  
# Tom says Meow!  
# ??? makes a sound.
```



DUCK TYPING (PYTHON STYLE)	FORMAL INTERFACES (OTHER LANGUAGES)
"If it has <code>.speak()</code> , treat it like something that speaks"	Must explicitly declare it implements an interface
Flexible, but errors surface at runtime	Stricter, errors caught earlier by the compiler

COMMON MISCONCEPTION

Polymorphism is not only about inheritance. In Python, any two unrelated objects that both implement `.speak()` can be used polymorphically — this is called *duck typing*: "if it walks like a duck and quacks like a duck...".

Try it: encapsulation guard

Deposit +50Deposit -20 (invalid)Reset

Balance: 100

Exercise 3.1 — Add a guarded setter

Extend `BankAccount` with a `withdraw(amount)` method that only allows withdrawal if `amount <= balance`, otherwise it should not change the balance.

```
def withdraw(self, amount):  
    if 0 < amount <= self.__balance:  
        self.__balance -= amount
```

Key takeaways

- Encapsulation controls access to internal state via methods/properties instead of direct attribute access.
- `@property` lets you expose a method as if it were a plain attribute.
- Polymorphism lets different classes respond to the same method call in their own way.
- Python favors duck typing over strict interface declarations.

Checkpoint Quiz — Application

4 questions · instant feedback · 70%+ to unlock Module 4

1. What is the main goal of encapsulation?

Making all code run faster

Bundling data with methods and controlling how that data is accessed/changed

Making every attribute public

Removing the need for classes

Why: Encapsulation bundles data and behavior together and restricts direct external access, so state changes happen through controlled methods/properties.

2. In Python, how is a 'private' attribute typically signaled?

private keyword

A leading underscore or double underscore, by convention/name-mangling

There is no way to signal this

ALL_CAPS naming

Why: Python uses a leading underscore (convention: 'internal use') or double underscore (name-mangled) rather than a strict private keyword like some other languages.

3. What does @property allow you to do?

Make a method callable like a plain attribute (e.g., obj.balance instead of obj.balance())

Turn any class into an abstract class

Automatically inherit from another class

Convert an object into JSON

Why: @property lets a method be accessed with attribute syntax (no parentheses), commonly used for controlled, read-only access to internal state.

4. What is 'duck typing'?

A strict interface requirement enforced at compile time

Treating an object as usable for an operation if it has the right method/attribute, regardless of its actual class

A method for typing faster in Python

A synonym for inheritance

Why: Duck typing means Python cares about whether an object supports the needed behavior (e.g. has `.speak()`), not what class it officially belongs to.

4/4 (100%)

Nice work — you've cleared this checkpoint.

Unlock Module 4 →

Module 04 Mastery — Dunder Methods, Composition & Design

Advanced

Dunder methods: hooking into Python's own syntax

"Dunder" (double-underscore) methods let your objects integrate with built-in syntax: printing, comparison, length, arithmetic, and more. You've already met `__init__` — here are more.

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __repr__(self):
        return f"Vector({self.x}, {self.y})"

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __eq__(self, other):
```

```

    return self.x == other.x and self.y == other.y

v1, v2 = Vector(1,2), Vector(3,4)
print(v1 + v2)      # Vector(4, 6)  - __add__ is called
print(v1 == v2)     # False         - __eq__ is called

```

DUNDER METHOD	TRIGGERED BY
<code>__repr__</code>	<code>print(obj)</code> , debugging output
<code>__add__</code>	<code>obj1 + obj2</code>
<code>__eq__</code>	<code>obj1 == obj2</code>
<code>__len__</code>	<code>len(obj)</code>
<code>__iter__</code>	<code>for x in obj</code>

Composition vs. inheritance

Composition builds objects out of other objects ("has-a") rather than extending a class ("is-a"). It's often more flexible than deep inheritance chains.

```

class Engine:
    def start(self):
        return "Engine roaring"

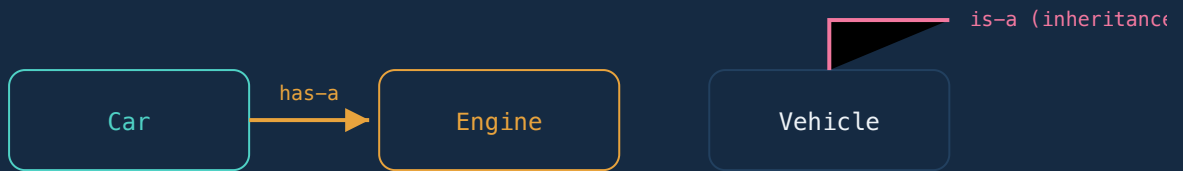
class Car:                                # Car HAS-A Engine (composition)
    def __init__(self):
        self.engine = Engine()

    def start(self):
        return self.engine.start()

```

ANALOGY

Inheritance is like being born into a family — you inherit traits you can't easily swap out. Composition is like hiring a specialist contractor: your Car doesn't need to *be* an Engine, it just needs *to have one* — and you can swap in a different engine without redesigning the car.



Abstract base classes: enforcing a contract

```

from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self): ...    # subclasses MUST implement this

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14159 * self.r ** 2

# Shape() -> TypeError: Can't instantiate abstract class
  
```

COMMON MISCONCEPTION

"Favor composition over inheritance" doesn't mean inheritance is bad — it means don't reach for inheritance by default. Use inheritance for genuine "is-a" relationships with shared behavior; use composition when you just need to reuse a capability.

Try it: vector addition

Vector(1,2)+Vector(3,4)

Vector(0,0)+Vector(5,-5)

Click to see `__add__` in action.

Exercise 4.1 — Implement `__len__`

Write a `Playlist` class holding a list of songs, implementing `__len__` so `len(playlist)` returns the number of songs.


```
class Playlist:
    def __init__(self, songs):
        self.songs = songs

    def __len__(self):
        return len(self.songs)
```

Key takeaways

- Dunder methods let your objects hook into Python's built-in syntax (printing, operators, length, iteration).
- Composition ("has-a") is often more flexible than deep inheritance ("is-a").
- Abstract base classes enforce that subclasses implement required methods.
- Good OOP design favors clarity and the right tool for the relationship, not maximal inheritance.

Checkpoint Quiz — Mastery

4 questions · instant feedback · 70%+ to unlock the Final Challenge

1. What triggers a class's `__add__` method?

Calling `len(obj)`

Using the `+` operator between two objects, e.g. `obj1 + obj2`

Printing the object

Comparing two objects with `==`

Why: `__add__` is invoked automatically when you use the `+` operator on instances of your class, letting you define custom addition behavior.

2. What best distinguishes composition from inheritance?

Composition builds objects out of other objects ('has-a'); inheritance extends a class ('is-a')

They are exactly the same thing

Composition requires an abstract base class

Inheritance can only be used once per program

Why: Composition ('has-a', e.g. Car has an Engine) builds functionality by holding other objects, while inheritance ('is-a') extends a class hierarchy.

3. What happens if you try to instantiate a class with an unimplemented `@abstractmethod`?

Python silently ignores it

Python raises a `TypeError`, since the class can't be instantiated

The method runs with default behavior

It only fails at the end of the program

Why: Abstract base classes with unimplemented `@abstractmethod` definitions raise a `TypeError` if you try to instantiate them directly — subclasses must implement the method first.

4. Why is 'favor composition over inheritance' good general guidance?

Because inheritance is always wrong

Because composition tends to be more flexible and avoids deep, tangled hierarchies where a genuine 'is-a' relationship doesn't exist

Because composition is faster to type

Because Python doesn't fully support inheritance

Why: It's guidance, not a hard rule — composition avoids fragile hierarchies when classes don't have a strict 'is-a' relationship, while inheritance still shines for genuine type hierarchies with shared behavior.

4/4 (100%)

Nice work — you've cleared this checkpoint.

Unlock Final Challenge →

Final Practical Challenge, Cheat Sheet & Where to Go Next

Final Practical Challenge

Design a small library system using everything from this tutorial:

- An abstract base class `LibraryItem` with an abstract method `info()`.
- Two subclasses, `Book` and `DVD`, each overriding `info()` (inheritance + polymorphism).
- A `Library` class that *has-a* list of items (composition) with a `__len__` dunder method.
- A private `__catalog_id` attribute on each item, exposed via a read-only `@property` (encapsulation).

```
# Main Program
library = Library()

book = Book("Python Basics", "Sachin", 101)
dvd = DVD("OOP Tutorial", 120, 102)

library.add_item(book)
library.add_item(dvd)

library.show_items()
```

✓ Looks solid — you've covered the key structural pieces (8/8 expected elements found).

Cheat Sheet

Defining a class

```
class Name:
    def __init__(self, ...): ...
```

Inheritance

```
class Child(Parent):
    def method(self):
        super().method()
```

Encapsulation

```
self.__private = x
@property
def private(self): return
self.__private
```

Polymorphism / duck typing

```
for obj in items:
    obj.speak() # each type responds
                differently
```

Common dunder

```
__init__ __repr__ __eq__
__add__ __len__ __iter__
```

Composition

```
class Car:
    def __init__(self):
        self.engine = Engine()
```

Summary Notes

- Classes are blueprints; objects are instances with their own state.
- Inheritance models "is-a" relationships and enables code reuse via overriding + `super()`.

- Encapsulation protects internal state behind methods/properties.
- Polymorphism lets different types share one interface — Python leans on duck typing.
- Dunder methods integrate your classes with Python's own syntax.
- Composition is often a more flexible alternative to deep inheritance chains.

Continue Learning

BOOKS

- "Fluent Python" — Luciano Ramalho
- "Python Object-Oriented Programming" — Dabbagh & Phillips
- "Clean Code" — Robert C. Martin (design principles)

DOCUMENTATION

- [Python docs: Classes](#)
- [Python data model \(dunder methods\)](#)
- [abc — Abstract Base Classes](#)

RESEARCH & DESIGN PAPERS

- Gamma et al., "Design Patterns" (Gang of Four)
- Liskov, "Data Abstraction and Hierarchy" (foundations of the Liskov Substitution Principle)

COMMUNITIES

- [r/learnpython](#)
- [Python Discord](#)
- [Stack Overflow — \[python\] \[oop\] tags](#)

PRACTICE PLATFORMS

- Exercism (Python track)
- LeetCode (design-pattern style problems)
- CodeWars (OOP kata)

SEARCH KEYWORDS

- "Python SOLID principles"
- "Python dataclasses vs classes"
- "Python metaclasses explained"
- "composition over inheritance python"

PROMPTS FOR FURTHER AI-ASSISTED LEARNING

- "Explain Python metaclasses with a real-world analogy and a minimal working example."
- "Show me how to refactor this class hierarchy using composition instead of inheritance."
- "Walk me through the SOLID principles using a Python example for each."

- "Compare Python's dataclasses to writing `__init__`/`__repr__`/`__eq__` by hand."



Mastery Certificate Unlocked

You've completed all four modules of **Object Kit: Python OOP**. Nice work.

Object Kit · Interactive Learning Studio · built for hands-on Python mastery